

1 Initial Intake Request

Onboarding is currently limited to a small group of Aza Plugins but we are opening to GA soon, and actively engaging teams building new Aza plugins to use Unified KB to get to market faster. Please complete this [intake form](#) to get started onboarding!

1.1 Required Information for Access Request

When completing the intake form, provide:

1. **Client Details:**
- Client service name (e.g., HRSC, MYHR, USC)
 - Business justification for access
 - Expected usage patterns and volume
2. **AWS Account Information:**
- List of AWS account IDs that need access by stage
 - Stage(s) required (Alpha, Beta, Gamma, PreProd, Prod)
 - IAM role names (if existing) that will assume the cross-account role
3. **Technical Contacts:**
- Primary technical contact and alias
 - Secondary contact for escalations
 - On-call rotation information (if applicable)
4. **Security & Compliance:**
- Data classification requirements
 - Any specific security requirements
 - Compliance frameworks that apply

2 Overview

The AET Content Knowledge Base Service (KBS) provides Model Context Protocol (MCP) access to Amazon's internal knowledge bases through a secure, multi-stage deployment architecture. This service enables clients to retrieve information from unified knowledge bases using standardized MCP protocols.

2.1 Available Stages

Stage	Purpose	Environment	ContentBus Mapping	Content	Sync Frequency
Alpha	Early development testing	Non-production	ContentBus Alpha	Synthetic/test data	Hourly
Beta	Integration testing	Non-production	ContentBus Beta	Synthetic/test data	Hourly
Gamma	Pre-production validation	Near-production	ContentBus Gamma	Synthetic/test data	Hourly
PreProd	Pre-production staging	Near-production	ContentBus Prod	Production content mirror	Hourly
Prod	Production workloads	Production	ContentBus Prod	Production	Hourly

The KBS MCP service implements **one-to-one stage mapping** with ContentBus (Unified Content Ingestion Pipeline) to ensure data consistency and operational simplicity.

 **Important:** Gamma does not contain production content. For pre-production testing with realistic content and results, use the PreProd stage. PreProd is a non-production account that mirrors production content from ContentBus.

3 Authentication Setup

3.1 IAM Access Setup

3.1.1 Cross-Account Role Pattern

Each client gets dedicated IAM roles following this naming convention:

```
kbs-mcp-role_{stage}_{client_id}
```

Example Role Names:

- kbs-mcp-role_Prod_hrsc
- kbs-mcp-role_Gamma_myhr
- kbs-mcp-role_Beta_usc

Role Permissions:

- execute-api: Invoke on the API Gateway endpoints
- Access to specific knowledge base resources based on client needs

3.1.2 IAM Policy Template

Clients need to create IAM policies allowing their services to assume the cross-account roles:

```
{  "Version": "2012-10-17",  "Statement": [    {      "Effect": "Allow",      "Action": "sts:AssumeRole",      "Resource": "arn:aws:iam::{KBS_ACCOUNT_ID}:role/kbs-mcp-role_{stage}_{client_id}"    }  ]}
```

3.1.3 API Gateway Authentication

The service uses AWS IAM (SigV4) authentication. Clients must:

- Assume the cross-account role
- Use temporary credentials to sign requests

4 v1 API

V1 is the latest version of the KBS MCP API. It introduces body-level target user specification, depth-limited filters, ACL filtering, and dual knowledge base retrieval.

All endpoints follow this pattern:

```
https://api.{region}.{stage}.knowledge.pxt.amazon.dev/iam/v1/mcp
```

5.2 Endpoints

Stage	Endpoint URL
Alpha	https://api.us-west-2.alpha.knowledge.pxt.amazon.dev/iam/v1/mcp
Beta	https://api.us-west-2.beta.knowledge.pxt.amazon.dev/iam/v1/mcp
Gamma	https://api.us-west-2.gamma.knowledge.pxt.amazon.dev/iam/v1/mcp
PreProd	https://api.us-west-2.preprod.knowledge.pxt.amazon.dev/iam/v1/mcp
Prod	https://api.us-west-2.prod.knowledge.pxt.amazon.dev/iam/v1/mcp

Server Name: knowledge-base-service · **Endpoint Path:** /iam/v1/mcp

5.3 Retrieve Tool

- Tool Name:** retrieve
- Purpose:** Retrieve content fragments from the unified and UCIP knowledge bases

Tool Usage Example

```
{  "method": "tools/call",  "params": {    "name": "retrieve",    "arguments": {      "query": "employee benefits information",      "maxResults": 10,      "targetUser": {        "LOGIN": "johndoe",        "PERSON_ID": "71c59248-d42f-45c6-a8d5-7c6d98ec4d9b"      },      "additionalFilters": {        "exactFilters": {          "equals": {            "key": "persona",            "value": "manager"          }        },        "partialFilters": {          "equals": {            "key": "plan_year",            "value": "2024"          }        },        "aclFilters": {          "equals": {            "key": "articleVisibility",            "value": "all-dls"          }        }      },      "meta": {        "progressToken": 0      }    }  } }
```

To skip personalization entirely, set targetUser to null:

```
{  "method": "tools/call",  "params": {    "name": "retrieve",    "arguments": {      "query": "general HR policies",      "maxResults": 5,      "targetUser": null    }  } }
```

targetUser Resolution Rules

- When targetUser is null, personalization filters (countryCode, stateCode, buildingCode, badgeColor, jobLevel, payRateType) are skipped entirely.
- When targetUser is omitted, the acting user (from x-acting-user header) is used as the target user.
- When both x-target-user header and targetUser body field are specified, their LOGIN must match. A mismatch results in an error.

Headers

```
{  "x-acting-user": "{\n  \"LOGIN\": \"<alias-for-eligibility>\",\n  \"x-target-user\": \"{\n    \"LOGIN\": \"<alias-for-applicability>\"\n  }\",\n  \"x-at0z-person-id\": \"<Person ID>\",\n  \"x-at0z-token-audience-type\": \"<at0z Personas>\",\n  \"x-amzn-transitive-authentication-token\": \"<TA Token>\" } } }
```

Note:

- x-acting-user is required. x-target-user is optional (can use targetUser in request body instead).
- x-at0z-token-audience-type (persona) is **required** for v1 requests.
- After following the [PIMA model](#) for parsing user identities, LOGIN (alias) is required.

Response Format

```
{  "_meta": {    "clientId": "{client_id}",    "requestId": "{uuid}"  },  "content": [    {      "type": "resource",      "resource": {        "uri": "...",        "mimeType": "text/plain",        "meta": {          "sourceUrl": "...",          "...        },        "text": "...",        "name": "...",        "title": "...",        "annotations": {          "audience": [            "user"          ],          "priority": "...",          "lastModified": "...        }      }    },    {      "type": "nextToken",      "nextToken": "{nextToken}"    }  ] }
```

Note: V1 response _meta does not include filters or nextToken fields (unlike v0).

4.3 Depth-Limited Filters

V1 enforces depth limits on filters to prevent unbounded recursion:

- exactFilters:** Maximum 2 layers of nesting (L0 primitives, L1 andAll/orAll of L0, L2 andAll of L1)
- partialFilters:** Maximum 1 layer of nesting (L0 primitives, L1 andAll/orAll of L0)
- acFilters:** Same depth limits as exactFilters (max 2 layers)

L0 (primitive operations): equals, notEquals, greaterThan, greaterThanOrEquals, lessThan, lessThanOrEquals, in, notIn, listContains, startsWith, stringContains

L1 (one layer of logical operators): andAll or orAll wrapping an array of L0 filters (minimum 2 conditions)

L2 (two layers, exactFilters/acFilters only): andAll wrapping an array of L1 filters (minimum 2 conditions). Note: orAll is not available at L2.

Example of a max-depth L2 exactFilter:

```
{  "exactFilters": {    "andAll": [      {        "andAll": [          {            "equals": {              "key": "persona",              "value": "manager"            }          },          {            "equals": {              "key": "articleVisibility",              "value": "all-dls"            }          }        ]      },      {        "orAll": [          {            "in": {              "key": "countryCode",              "value": ["US", "CA"]            }          },          {            "equals": {              "key": "region",              "value": "AMER"            }          }        ]      }    ]  } }
```

5 v0 API (Deprecated)

 v0 is deprecated. New integrations should use the v1 API above.

[Show v0 documentation](#)

6 Client Configuration

6.1 TypeScript/Node.js Implementation

6.1.1 Required Dependencies

```
npm install @modelcontextprotocol/sdk @aws-sdk/credential-providers aws-sigv4-fetch
```

6.1.2 Client Setup Example

```
import { Client } from '@modelcontextprotocol/sdk/client/index.js';import { StreamableHTTPClientTransport } from '@modelcontextprotocol/sdk/client/http.js';import { fromTemporaryCredentials } from '@aws-sdk/credential-providers';import { createSignedFetcher } from 'aws-sigv4-fetch';// Configure ARNconst ROLE_ARN = 'arn:aws:iam::${KBS_ACCOUNT_ID}:role/kbs-mcp-role_${Stage}_${client_id}';const ENDPOINT = 'https://api.us-west-2.stage.knowledge.pxt.amazon.dev/iam/v1/mcp';const REGION = 'us-west-2';// Create MCP client with AWS SigV4 authenticationexport const createKbsMcpClient = async ({  employeeLogin,  concernedEmployeeLogin,  personId,  persona,  transitiveAuthToken,}) => Promise.resolve({  // Get temporary credentials by assuming the cross-account role  const credentials = fromTemporaryCredentials({    params: {      RoleArn: ROLE_ARN,      // Optional Session Name      RoleSessionName: 'kbs-mcp-session',    },  });  // Create AWS SigV4 signed fetch  const signedFetch = createSignedFetcher({    credentials,    region: REGION,    service: 'execute-api',  });  // Create HTTP transport with authentication  const transport = new StreamableHTTPClientTransport(ENDPOINT, {    fetch: signedFetch,    requestInit: {      'x-acting-user': JSON.stringify({ LOGIN: employeeLogin }),      // Optional Target if different from Actor      'x-target-user': JSON.stringify({ LOGIN: concernedEmployeeLogin }),      'x-at0z-person-id': personId, // Include for future support      'x-at0z-token-audience-type': persona,      'x-amzn-transitive-authentication-token': transitiveAuthToken,    },  });  // Create and return MCP client  const client = new Client({    name: 'knowledge-base-service',    version: '0.1.0',  });  await client.connect(transport);  return client;});
```

6.1.3 Usage Example

```
// Use the client to call the Retrieve toolconst callRetrieveTool = async (query: string, maxResults: number = 10) => {  const client = await createKbsMcpClient({ employeeLogin });  try {    const response = await client.callTool({      name: 'retrieve',      arguments: {        query,        maxResults,      },    });    return response;  } catch (error) {    console.error('Error calling Retrieve tool:', error);    throw error;  } finally {    await client.close();  }  };// Example usageconst results = await callRetrieveTool('employee benefits information', 5);console.log('Retrieved results:', results);
```

6.1.4 Step-by-Step Setup

- Receive Cross-Account Role ARN** from the KBS team after access approval
- Configure IAM Policy** in your AWS account to assume the role
- Install Required Dependencies** (@modelcontextprotocol/sdk, @aws-sdk/credential-providers, aws-sigv4-fetch)
- Implement MCP Client** using the TypeScript example above
- Test Connectivity** using the appropriate stage endpoint
- Implement Error Handling** and retry logic for production use

7 Support Information

7.1 Getting Help

For access requests, issues, or questions:

Primary Support: SIM Tickets

- CTY:** PXT/AETContentOptimization/Support
- Resolver Group:** AET ML Engineering Seattle
- Critical Issues:** Use Sev-2 SIM tickets

7.2 Common Issues & Solutions

7.2.1 Authentication Failures

- Symptoms:** 403 Forbidden responses
- Solutions:** Verify cross-account role ARN, check IAM policy attachments, validate STS assume role permissions

7.2.2 Timeout Issues

- Symptoms:** Request timeouts or 504 Gateway Timeout
- Solutions:** Implement retry logic with exponential backoff, check client timeout configurations, verify network connectivity to us-west-2

7.2.3 Rate Limiting

- Symptoms:** 429 Too Many Requests
- Solutions:** Implement client-side rate limiting, use jitter in retry logic, consider request batching where applicable

8 Getting Started Checklist

- Complete Taskel intake form for access request
- Provide AWS account IDs and stage requirements
- Receive cross-account role ARN from KBS team
- Configure IAM policies in client account
- Implement MCP client using TypeScript example
- Test connectivity with appropriate stage endpoint
- Implement error handling and retry logic
- Document internal procedures for your team